

TAGD-03-ai

Filippo Visconti

Contents

1	Consistenza nelle basi di dati distribuite	2
1.1	Partizionamento e Replicazione	2
1.2	Sistemi distribuiti: il trade-off tra Safety e Liveness	2
1.3	Il Teorema CAP	2
1.4	Il problema del Consenso	3
1.5	L'algoritmo RAFT	4
2	Quality of Service (QoS) nelle Architetture per la Gestione dei Dati	5
2.1	Attributi di Qualità nei Sistemi IT	5
2.2	Disponibilità (Availability)	6
2.3	Scalabilità (Scalability)	7
2.4	Il Ciclo di Vita del Sistema e la QoS	7
3	Modellamento di sistemi: aspetti qualitativi	7
3.1	Competizione per le risorse e modellamento	7
3.2	Modellamento di un server database semplice	8
3.3	Classi multiple di workload	8
3.4	Service Level Agreement (SLA)	9
3.5	Admission Control	10
3.6	Discipline di accodamento	10
3.7	Class Switching	10
4	Modellamento di sistemi: reti di code (QN)	10
4.1	Parametri di input e caratterizzazione del workload	10
4.2	Notazione formale e metriche di performance	11
4.3	Modellamento di un database server: esempio pratico	11
4.4	Analisi di workload batch e scenari di sistema	11
5	Modellamento di sistemi: aspetti quantitativi	12
5.1	Variabili operazionali e definizioni fondamentali	12
5.2	Esempio pratico di analisi operazionale	13
5.3	Leggi operazionali fondamentali	13
5.4	Legge di Little (Little's Law)	14
6	Legge del tempo di risposta interattivo e algoritmi di analisi	15
6.1	Esempi applicativi delle leggi operazionali	15
6.2	Riepilogo delle leggi fondamentali (In sintesi)	15
6.3	Esercitazione pratica sui modelli di code	15
6.4	Mean Value Analysis (MVA)	16

1 Consistenza nelle basi di dati distribuite

I moderni sistemi di gestione di basi di dati (DBMS) devono affrontare la necessità di scalare per supportare i requisiti prestazionali e di carico delle applicazioni contemporanee. Esistono due direttrici principali per l'espansione: la scalabilità verticale (scale up) e la scalabilità orizzontale (scale out). La scalabilità verticale prevede il potenziamento del singolo server attraverso l'allocazione di risorse hardware superiori, quali CPU più potenti, una maggiore quantità di memoria primaria e dischi di capacità elevata; tuttavia, questa strategia incontra un limite fisico insormontabile nella quantità massima di risorse che possono essere concentrate in una singola macchina. La scalabilità orizzontale prevede invece l'aumento del numero di server fisici impiegati, utilizzando tecniche di partizionamento (sharding) e replicazione. In questo scenario, il limite principale è rappresentato dal sovraccarico di comunicazione (communication overhead) tra i nodi, ma non è l'unico vincolo.

1.1 Partizionamento e Replicazione

L'architettura distribuita si fonda su due concetti chiave che possono essere combinati tra loro:

- **Partizionamento:** consiste nella suddivisione logica e fisica dei dati tra i vari nodi del sistema.
- **Replicazione:** prevede la memorizzazione degli stessi dati su più nodi differenti.

Uno schema esemplificativo vede una configurazione con sei macchine interconnesse. In un sistema puramente partizionato, la Machine 1 ospita i Chunk 1 e 2, la Machine 2 ospita i Chunk 3 e 4, e la Machine 3 ospita i Chunk 5 e 6. Quando si introduce la replicazione, i medesimi dati vengono duplicati su altre macchine: la Machine 4 replica il contenuto della Machine 1 (Chunk 1 e 2), la Machine 5 replica la Machine 2 (Chunk 3 e 4), e la Machine 6 replica la Machine 3 (Chunk 5 e 6).

Mantenere la consistenza tra le repliche di una base di dati distribuita rappresenta una sfida complessa. Si consideri un esempio con due repliche del database dove il saldo iniziale è $Bal = 1000$. Vengono generati due eventi concorrenti: l'Evento 1 aggiunge 1000 € e l'Evento 2 aggiunge un interesse del 5%. Se sulla prima replica viene eseguito l'ordine (1, 2), il saldo finale sarà $(1000 + 1000) \times 1.05 = 2100$. Se sulla seconda replica viene eseguito l'ordine (2, 1), il saldo finale sarà $(1000 \times 1.05) + 1000 = 2050$. Questa divergenza evidenzia la difficoltà nel garantire la coerenza tra le copie.

1.2 Sistemi distribuiti: il trade-off tra Safety e Liveness

In un'elaborazione distribuita operante in un contesto non affidabile (unreliable), è impossibile garantire simultaneamente le proprietà di safety e liveness.

- **Safety:** descrive il principio per cui “non succede niente di male”. La consistenza, intesa come coerenza tra le copie, è una proprietà di safety: tutte le risposte fornite ai client devono essere corrette secondo una determinata nozione di correttezza.
- **Liveness:** descrive il principio per cui “prima o poi succede qualcosa di buono”. La disponibilità (availability) è una proprietà di liveness: ogni richiesta effettuata da un client deve ricevere, prima o poi, una risposta.
- **Unreliability:** indica la presenza di fallimenti intrinseci, quali partizionamenti della rete (network partitioning), crash dei nodi (crash failures) o comportamenti malevoli dei nodi (Byzantine failures).

1.3 Il Teorema CAP

Il teorema CAP (introdotto da Eric Brewer) formalizza il trade-off tra tre proprietà in un sistema distribuito composto da un insieme di nodi $G_1, \dots, G_n$ distribuiti geograficamente, dove i client eseguono operazioni di lettura e scrittura:

1. **Consistenza (Atomic consistency):** definita dalla semantica complessiva del servizio. Le sequenze di operazioni devono produrre lo stesso effetto che avrebbero se fossero eseguite su un singolo server centralizzato. Il client deve percepire il servizio come atomico e le risposte devono essere coerenti.
2. **Disponibilità (Availability):** ogni richiesta riceve una risposta (eventual response). Una risposta eccessivamente lenta può essere considerata errata o equivalente a un fault.

3. **Tolleranza al partizionamento (Network partitioning):** descrive la suddivisione della rete in gruppi che non possono comunicare tra loro. Poiché i messaggi possono essere ritardati o persi, l'inaffidabilità della comunicazione è un dato di fatto del sistema.

L'enunciato del teorema CAP stabilisce che in un network soggetto a partizionamento, è impossibile implementare una memoria condivisa read/write atomica che fornisca risposta a ogni richiesta. La dimostrazione intuitiva prevede che se due nodi sono separati da un partizionamento, un aggiornamento su uno non può essere propagato all'altro. Di conseguenza, l'altro nodo potrà o non rispondere (violando la disponibilità) o rispondere con un dato obsoleto (violando la consistenza).

1.3.1 Implicazioni pratiche e compromessi

Nella costruzione di sistemi reali si adottano diversi compromessi:

- **Best effort availability:** se la consistenza è un vincolo inderogabile, la disponibilità passa in secondo piano. Un esempio è il Google Lock Service (Chubby), che supporta GFS e Big Table. Fornisce consistenza forte tramite un design primary-backup; se i server sono partizionati, il servizio diventa non disponibile.
- **Best effort consistency:** se la disponibilità e la velocità di risposta sono prioritarie, si tollerano inconsistenze temporanee. Esempi includono il caching di contenuti web (Akamai) e i sistemi basati su eventual consistency.

1.3.2 Consistenza basata su Quorum

Un meccanismo per bilanciare consistenza e disponibilità è l'uso dei quorum:

- N_W : numero minimo di nodi che devono confermare una scrittura.
- N_R : numero minimo di nodi che devono confermare una lettura.
- La condizione $N_R + N_W > n$ garantisce letture fortemente consistenti (strongly consistent reads), poiché l'intersezione tra l'insieme di lettura e quello di scrittura non è mai vuota.
- La condizione $N_W > n/2$ serve a evitare conflitti scrittura-scrittura, garantendo l'isolamento.

I trade-off possono essere definiti dinamicamente:

- **Soglie di “out-of-date”-ness:** si tollerano inconsistenze temporanee per limitare la non disponibilità solo a partizionamenti molto lunghi.
- **Stato dell'esecuzione:** in una prenotazione aerea, si può tollerare l'inconsistenza quando ci sono molti posti disponibili, ma la consistenza deve diventare stretta quando i posti rimasti sono pochi.
- **Caratteristiche dei dati:** l'inventario di un e-commerce può essere disallineato, ma il carrello degli acquisti deve essere coerente.
- **Tipo di operazione:** le letture possono tollerare inconsistenze (es. navigazione catalogo), mentre le scritture no (es. acquisto prodotto).
- **Utente o Gerarchia:** gli utenti possono essere partizionati geograficamente con consistenza maggiore entro la propria partizione. Scendendo in un partizionamento gerarchico, si possono fornire livelli di consistenza più elevati.

1.4 Il problema del Consenso

Il consenso riguarda la capacità di un insieme di nodi $G_1, \dots, G_n$, ciascuno con un valore iniziale v_i , di accordarsi su un unico valore di output. Le proprietà richieste sono:

- **Agreement:** tutti i nodi devono restituire lo stesso valore (safety).
- **Validity:** il valore di output deve essere stato proposto come input da almeno un nodo (safety).
- **Termination:** ogni nodo deve prima o poi restituire un valore (liveness).

Gli algoritmi di consenso assicurano la safety ritornando sempre risultati corretti e garantiscono la liveness se la maggioranza dei nodi è attiva, anche in presenza di ritardi, partizionamenti o riordinamento di messaggi.

1.4.1 Macchine a Stati Replicate (Replicated State Machine)

Gli algoritmi di consenso si basano sul concetto di macchina a stati replicata. Ogni nodo possiede una copia identica di una macchina a stati e opera su un log replicato (replicated log), ovvero una sequenza di

istruzioni. Poiché le macchine sono deterministiche, se applicano gli stessi comandi nello stesso ordine dal medesimo log, raggiungeranno lo stesso stato. L'algoritmo di consenso ha la responsabilità di mantenere allineati i log tra i nodi attraverso un modulo di consenso che riceve i comandi dai client e comunica con gli altri nodi per far convergere i log.

1.5 L'algoritmo RAFT

RAFT è un algoritmo di consenso introdotto da Leslie Lamport nel 1989 (come versione semplificata e didattica di Paxos) per essere più comprensibile e adatto a sistemi reali. Il problema del consenso in RAFT viene decomposto in tre sottoproblemi:

1. **Elezione del leader:** scelta di un nuovo leader al fallimento del precedente.
2. **Replicazione del log:** il leader accetta le entry dai client e le replica nel cluster.
3. **Safety:** garanzia che se un nodo applica un'operazione, nessun altro nodo applicherà un comando diverso per la stessa posizione.

1.5.1 Stati e tempo in RAFT

Un cluster RAFT solitamente contiene 5 server per tollerare 2 fallimenti simultanei. Ogni server può trovarsi in uno di tre stati:

- **Leader:** gestisce le richieste dei client. Se un client contatta un follower, viene reindirizzato al leader.
- **Follower:** stato passivo; risponde alle richieste dei leader o dei candidate.
- **Candidate:** stato temporaneo utilizzato durante le elezioni.

Il tempo è suddiviso in “terms” di lunghezza arbitraria, ognuno dei quali inizia con un'elezione. I term agiscono come un orologio logico (logical clock) per individuare informazioni obsolete. Se un server scopre di avere un progressivo di term inferiore a un altro, si aggiorna; se un leader o un candidate scopre un term superiore, torna immediatamente allo stato di follower. Le richieste riferite a term superati vengono rigettate.

1.5.2 Comunicazione e Invarianti

La comunicazione avviene tramite due tipi di RPC (Remote Procedure Call):

- **RequestVOTE:** avviata dai candidati durante le elezioni.
- **AppendEntries:** avviata dai leader per replicare le entry del log e fungere da heartbeat.

RAFT si fonda su rigide invarianti per mantenere l'allineamento:

- **Election safety:** al massimo un leader per ogni term.
- **Leader append-only:** il leader non sovrascrive né cancella mai le proprie entry.
- **Log matching:** entry con stesso indice e stesso term implicano comandi identici e log identici nelle posizioni precedenti.
- **Leader completeness:** le entry committate in un term sono presenti nei leader di tutti i term successivi.
- **State machine safety:** nessuna macchina a stati applicherà mai comandi diversi per lo stesso indice.

1.5.3 Processo di Elezione e Replicazione

Un server inizia come follower. Se non riceve heartbeat entro un “election timeout”, diventa candidate, incrementa il term, vota per se stesso e invia RequestVote. Vince chi ottiene la maggioranza dei voti nel cluster. Per evitare split vote infiniti (dove nessuno ottiene la maggioranza), si usa un “randomized election timeout”.

Una volta eletto, il leader riceve comandi dai client e li aggiunge al proprio log. Invia AppendEntries in parallelo. Una entry è considerata “committed” quando è replicata sulla maggioranza dei server. Il commit di una entry implica il commit automatico di tutte le entry precedenti nel log. Il leader include l'indice committato più alto in ogni AppendEntries per informare i follower.

In caso di inconsistenze (disallineamento dei log dovuto a crash), il leader forza i follower a uniformarsi al suo log. Individua l'ultima entry coincidente tramite un consistency check, cancella le entry successive nel follower e invia le proprie. Il leader mantiene un `nextIndex` per ogni follower per gestire questo riallineamento.

1.5.4 Restrizioni e Safety

Per garantire la safety, RAFT impone una restrizione all'elezione: un votante nega il voto a un candidato se il log del candidato è meno aggiornato del proprio. L'aggiornamento si confronta controllando prima il numero del term e poi l'indice dell'ultima entry. Inoltre, un leader non effettua mai il commit di una entry di un term precedente contando solo le repliche; il commit avviene solo per le entry del term corrente, trascinando con sé per log matching le entry precedenti.

1.5.5 Disponibilità e Timing

Mentre la safety è indipendente dal tempo, la disponibilità dipende dal rispetto della relazione: $broadcastTime \ll electionTimeout \ll MTBF$ (Mean Time Between Failures). Se i messaggi sono troppo lenti rispetto ai crash, il sistema non riuscirà a eleggere un leader stabile e non farà progressi.

2 Quality of Service (QoS) nelle Architetture per la Gestione dei Dati

I moderni sistemi IT e i servizi basati su Internet sono diventati pervasivi, esponendosi a una platea di utilizzatori potenzialmente illimitata. In fase di progettazione, non è più sufficiente concentrarsi esclusivamente sulle funzionalità del sistema, ma è imperativo considerare gli aspetti di qualità, definiti anche requisiti non funzionali. Tra questi, assumono un'importanza critica le prestazioni, la disponibilità, la sicurezza e la manutenibilità. Esistono settori specifici in cui la qualità del servizio (Quality of Service, QoS) non è solo un parametro di efficienza, ma un requisito fondamentale per la sicurezza e l'operatività; si pensi ai sistemi per le chiamate di emergenza (112), alle piattaforme di trading online dove il tempo è denaro, o alle applicazioni mediche salvavita.

2.1 Attributi di Qualità nei Sistemi IT

L'analisi della QoS si focalizza su quattro attributi principali che permettono di misurare l'efficienza e l'affidabilità di un sistema: il tempo di risposta, il throughput, la disponibilità e la scalabilità.

Per comprendere dove si generano i ritardi, è necessario analizzare il percorso di una richiesta in un'architettura tipica, come quella di un e-commerce. Il tempo totale percepito dall'utente è la somma di tre macro-componenti:

1. **Browser Time:** include il tempo di elaborazione locale (Processing) e le operazioni di input/output (I/O) sul dispositivo dell'utente.
2. **Network Time:** comprende il transito dal browser all'Internet Service Provider (ISP), il tempo di percorrenza sulla rete Internet globale e il passaggio finale dall'ISP al server di destinazione.
3. **E-commerce Server Time:** include l'elaborazione interna al server (Processing), l'accesso ai dati (I/O) e i ritardi di networking lato server. In ognuna di queste fasi può innescarsi il fenomeno della congestione, che degrada le prestazioni complessive.

2.1.1 Tempo di Risposta (Response Time)

Il tempo di risposta è definito come l'intervallo temporale necessario a un sistema per reagire a una specifica richiesta. Un esempio classico è il tempo che intercorre tra l'inserimento di un indirizzo nella barra del browser e l'effettiva apparizione della pagina web richiesta.

2.1.2 Throughput

Il throughput rappresenta il numero di richieste evase con successo dal sistema nell'unità di tempo, ed è spesso indicato come "tasso" (rate). L'unità di misura è strettamente legata alla natura del sistema analizzato:

- **Sistemi OLTP (On Line Transaction Processing):** misurato in transazioni al secondo (*tps*).
- **Siti Web:** misurato in richieste HTTP al secondo, visualizzazioni di pagina al secondo (*PageViews/sec*) o byte trasferiti al secondo (*Bytes/sec*).
- **Router di rete:** misurato in pacchetti al secondo (*pps*) o megabyte trasferiti al secondo.
- **CPU:** misurato in milioni di istruzioni al secondo (*MIPS*) o operazioni in virgola mobile al secondo (*FLOPS*).
- **Dispositivi di storage (Dischi):** misurato in operazioni di I/O al secondo (*IOPS*) o kilobyte trasferiti al secondo.

Calcolo del Throughput Massimo e Relazione con il Carico Si consideri un disco in un sistema OLTP in cui un'operazione di I/O impiega mediamente $10ms$ ($0.01s$). Se il disco è costantemente occupato, ovvero la sua utilizzazione è pari al 100%, il throughput massimo è calcolabile come: $Throughput_{max} = \frac{1}{0.01s} = 100 \text{ IOPS}$

Se il carico di lavoro (workload) genera un tasso di richieste di 60 IOPS, l'utilizzazione della risorsa sarà proporzionalmente del 60%. In una prima approssimazione, possiamo stabilire che: $throughput = \min(capacit, workload)$

Tuttavia, se il workload supera la capacità nominale (ad esempio 150 IOPS a fronte di una capacità di 100), il sistema entra in saturazione. In molti sistemi reali, superata la soglia di saturazione, si verifica il fenomeno del **thrashing**: invece di stabilizzarsi sul throughput massimo (andamento “no thrashing”), le prestazioni crollano drasticamente a causa dell'eccessivo overhead di gestione delle code e dei conflitti tra le richieste.

2.2 Disponibilità (Availability)

La disponibilità è definita come la frazione di tempo durante la quale il sistema risulta attivo, integro e accessibile per i suoi utenti. Una bassa disponibilità non comporta solo una perdita immediata di transazioni o clienti, ma mina gravemente la credibilità e la reputazione del fornitore del servizio.

La disponibilità viene spesso espressa in termini di “nove” (es. 99.99%). Se un sistema dichiara una disponibilità del 99.99% su un periodo di 30 giorni, il tempo totale di indisponibilità ammesso si calcola come: $Indisponibilit = (1 - 0.9999) \times 30 \text{ giorni} \times 24 \frac{hr}{giorno} \times 60 \frac{min}{hr} = 4.32 \text{ minuti}$

2.2.1 Esercizio Pratico di Calcolo della Disponibilità

Si analizzi un sistema e-commerce che in due giorni ha subito i seguenti periodi di fermo:

- **Giorno 1:** 12 min (ore 1:25 AM), 1 min (ore 7:01 AM), 5 min (ore 8:31 PM).
- **Giorno 2:** 10 min (ore 2:15 AM), 6 min (ore 9:12 PM). Il totale dei minuti di indisponibilità è $12 + 1 + 5 + 10 + 6 = 34$ minuti. La disponibilità a su 2 giorni ($2 \times 24 \times 60 = 2880$ minuti) è: $a = 1 - \frac{34}{2880} = 0.988194$ (circa 98.82%).

2.2.2 Cause di Indisponibilità

Le interruzioni di servizio derivano principalmente da tre fattori:

1. **Guasti Hardware:** rottura fisica di componenti, come il malfunzionamento di uno switch di rete o di un disco.
2. **Crash Software:** errori di programmazione (bug) che si attivano durante l'esecuzione di specifici casi d'uso.
3. **Sovraccarichi:** il sistema è integro, ma le richieste in ingresso superano le risorse disponibili. Ciò porta al respingimento delle richieste eccedenti (ad esempio, un DBMS che rifiuta nuove connessioni perché il pool è saturo).

Per gestire i sovraccarichi e prevenire il degrado incontrollato dei tempi di risposta, si utilizza l'**Admission Control**. Senza di esso, il tempo di risposta cresce esponenzialmente al crescere del carico; con l'Admission Control, il sistema accetta solo il carico che può gestire efficientemente, mantenendo i tempi di risposta entro soglie accettabili per le richieste accolte.

2.3 Scalabilità (Scalability)

Un sistema è definito scalabile se le sue prestazioni non degradano significativamente all'aumentare del numero di utenti o del carico di lavoro. In un grafico che mette in relazione il tempo di risposta con il carico, un sistema scalabile (Sistema B) mostra una crescita contenuta o lineare, mentre un sistema non scalabile (Sistema A) mostra un'impennata dei tempi di risposta non appena il carico aumenta minimamente.

2.4 Il Ciclo di Vita del Sistema e la QoS

È un errore comune, ma rischioso, valutare le prestazioni solo dopo la messa in produzione. Questo approccio comporta costi elevati per hardware aggiuntivo o sessioni estenuanti di fine-tuning, e nei casi peggiori può richiedere la riprogettazione totale del software. Lo studio della QoS deve quindi permeare ogni fase del ciclo di vita:

1. **Requirements Analysis and Specification:** I requisiti devono essere distinti in **funzionali** (es. “il sistema deve permettere la ricerca per ISBN”) e **non funzionali** (es. “il sistema deve gestire 50 ricerche/sec con risposta entro 2s nel 95% dei casi”).
2. **System Design:** Progettazione di architetture e algoritmi. Il riuso di componenti esistenti accelera lo sviluppo funzionale, ma non garantisce il rispetto dei requisiti di performance.
3. **System Development:** Lo sviluppatore deve testare le prestazioni anche in fase di codice (es. evitare join complessi che funzionano su pochi record ma falliscono su milioni di righe). È fondamentale instrumentare il codice per facilitare la raccolta dati.
4. **Testing:** Oltre agli unit test, sono necessari i **load test** per simulare l'interazione contemporanea di molti utenti (modelli a ciclo aperto o chiuso).
5. **Deployment:** Fase di fine-tuning dei parametri di configurazione per ottimizzare i risultati basandosi su modelli predittivi.
6. **Operation and Evolution:** Monitoraggio costante del sistema in produzione.

2.4.1 Monitoraggio delle Prestazioni

Durante l'esercizio del sistema, è necessario monitorare diverse categorie di metriche:

- **Workload:** periodi di picco, caratteristiche dei processi di arrivo (bursty?), presenza di anomalie (es. Denial of Service), identificazione delle classi di richieste.
- **Metriche Esterne:** tempi di risposta (media, deviazione standard, percentili), throughput e tasso di scarto delle richieste. Bisogna ricordare che la percezione dell'utente è influenzata anche dalla sua connessione e posizione geografica.
- **Metriche Interne:** utilizzazione di CPU, memoria, storage e rete; lunghezza delle code hardware e software. È fondamentale filtrare i dati per non essere sopraffatti da informazioni irrilevanti.
- **Disponibilità:** verificata tramite agenti esterni che effettuano richieste casuali. Il test stesso non deve però appesantire il sistema (non deve diventare workload).

Strumenti come **iostat** su Linux permettono di visualizzare in tempo reale l'utilizzazione dei dispositivi di storage, indicando il numero di letture e scritture al secondo ($r/s, w/s$), i KB trasferiti, il tempo medio di attesa ($r_a wait$) e la dimensione media delle richieste ($rareq - sz$). Su sistemi Windows, funzionalità analoghe sono fornite dal **Performance Monitor**.

3 Modellamento di sistemi: aspetti qualitativi

La progettazione delle prestazioni di un sistema informativo è un'attività critica che richiede un framework strutturato. Il punto di partenza risiede nell'idea che i sistemi informatici siano composti da un insieme di risorse fisiche e logiche, quali processori (CPU), dischi magnetici o a stato solido, canali di comunicazione, thread di processi, sezioni critiche del codice e lock sui database. Queste risorse sono intrinsecamente condivise tra le varie richieste a cui il sistema è soggetto durante il suo esercizio.

3.1 Competizione per le risorse e modellamento

Poiché le risorse sono condivise, le richieste inviate al sistema entrano in competizione per accedervi. Questo fenomeno genera inevitabilmente la creazione di code quando la risorsa desiderata è occupata. Il

framework di analisi si basa quindi sui modelli di code (queuing models). Dato che ogni singola risorsa possiede la propria coda specifica, l'intero sistema viene rappresentato come una rete di code (queuing networks).

Un modello di un sistema è un'astrazione della realtà. Il livello di dettaglio e gli aspetti da includere dipendono strettamente dallo scopo dell'analisi. Ad esempio, se si deve valutare un sistema destinato esclusivamente ad applicazioni CPU-intensive, potrebbe non essere necessario modellare le prestazioni dei dischi. È fondamentale ricordare che l'unico modello di un sistema completamente affidabile è una copia esatta del sistema stesso, soluzione tuttavia estremamente costosa e difficile da gestire. Per questo motivo si ricorre a modelli simulativi o analitici.

3.1.1 Modelli Simulativi

I modelli simulativi si basano su programmi software che riproducono il comportamento dei diversi componenti del sistema. Il carico di lavoro (workload) può essere riprodotto partendo da una traccia reale, da un benchmark sintetico o generato seguendo specifiche distribuzioni di probabilità. All'interno del simulatore, i componenti sono arricchiti con contatori per le metriche di prestazione. Al termine della simulazione, questi dati vengono usati per calcolare statistiche. Ad esempio, il tempo di risposta medio di un componente si ottiene tramite la formula $T = \frac{\sum_{i=1}^{nt} T_i}{nt}$, dove T rappresenta il tempo di risposta medio, T_i è il tempo di risposta della singola richiesta i e nt è il numero totale di richieste processate. Questi modelli permettono studi molto dettagliati e accurati, ma sono costosi e complessi da realizzare proprio a causa dell'alto livello di dettaglio richiesto.

3.1.2 Modelli Analitici

I modelli analitici sono costituiti da insiemi di formule matematiche o algoritmi che calcolano le misure di prestazione in funzione del workload. Per rimanere matematicamente trattabili, sono solitamente meno dettagliati dei modelli simulativi e, di conseguenza, possono risultare meno accurati. Tuttavia, sono estremamente più efficienti da eseguire e richiedono parametri di input più semplici da reperire grazie al loro maggior livello di astrazione.

Nella pratica ingegneristica e nel capacity planning si adotta spesso un approccio ibrido: si utilizza inizialmente un modello simulativo per verificare la validità di un modello analitico. Se i risultati sono soddisfacenti, si prosegue con il modello analitico, accettando un margine d'errore che in questo ambito può arrivare fino al 30% senza inficiare la bontà della pianificazione.

3.2 Modellamento di un server database semplice

Si consideri un server database caratterizzato da una singola CPU e un singolo disco, con un tasso di arrivo delle transazioni pari a 1.5 transazioni al secondo (tps). Durante l'esecuzione, una transazione utilizza in modo alternato le risorse di CPU e disco, spesso ripetendo questo ciclo più volte.

In una rete di code (QN), le risorse sono rappresentate come "centri di calcolo" dotati di una coda. Graficamente, un centro di servizio singolo (a) è composto da un rettangolo suddiviso a scomparti (la waiting line) seguito da un cerchio (il servente); l'unione di questi due elementi costituisce la coda (queue). Un centro multi-servente (b) presenta una singola waiting line che alimenta m cerchi (serventi) paralleli.

Lo scheletro del modello per il DB server prevede un flusso circolare: le transazioni in arrivo (arriving transactions) entrano nella coda della CPU. Una volta servite dalla CPU, le transazioni possono uscire dal sistema (completing transactions) oppure dirigersi verso la coda del disco. Dopo essere state servite dal disco, esse rientrano nella coda della CPU per proseguire l'elaborazione. Mappare un sistema reale su questo schema non è banale e richiede di decidere quali aspetti modellare per rispondere a domande specifiche, come la variazione del tempo di risposta al variare del tasso di arrivo o alla sostituzione dell'hardware.

3.3 Classi multiple di workload

L'analisi dei log del database spesso rivela transazioni con caratteristiche profondamente diverse. Per migliorare l'accuratezza del modello, è opportuno raggruppare le transazioni in classi di servizio (ad esempio: semplici, medie, complesse) basandosi sulla combinazione del tempo di CPU e del numero di operazioni di I/O.

Si consideri il seguente esempio di ripartizione del carico:

- **Classe Semplice:** 45% del totale, tempo medio CPU 0.04s, numero medio I/O 5.5.
- **Classe Medio:** 25% del totale, tempo medio CPU 0.18s, numero medio I/O 28.9.
- **Classe Complesso:** 30% del totale, tempo medio CPU 1.20s, numero medio I/O 85.0.

L'adozione di un modello QN multi-classe è necessaria quando:

1. Esistono valori eterogenei di **service demand**, ovvero il tempo totale medio speso da una transazione in una specifica risorsa.
2. Sono presenti tipi diversi di workload (es. transazioni online brevi vs processamenti batch pesanti per reportistica).
3. Esistono differenti **Service Level Objectives (SLO)**, come limiti superiori al tempo di risposta differenziati (es. 1.2s per le semplici, 2.5s per le medie, 8s per le complesse).

3.3.1 Classi Aperte

Una classe di workload è definita aperta se la sua intensità è specificata tramite un tasso di arrivi. In questo caso, il numero di clienti nel sistema è potenzialmente illimitato e il tasso di arrivo è solitamente indipendente dal numero di transazioni già in gestione. In equilibrio, il throughput è uguale al tasso di arrivo. Riprendendo l'esempio precedente con un tasso totale di 1.5 tps:

- Tasso classe “semplice”: $1.5 \times 0.45 = 0.675$ tps
- Tasso classe “medio”: $1.5 \times 0.25 = 0.375$ tps
- Tasso classe “complesso”: $1.5 \times 0.3 = 0.45$ tps

3.3.2 Classi Chiuse

Una classe è definita chiusa quando la sua intensità è specificata dalla popolazione fissa all'interno del sistema. Si pensi a dei job batch eseguiti di notte: se la popolazione è fissata a 5, non appena un job termina, un nuovo job entra nel sistema, mantenendo il numero di richieste concorrenti costante. Qui il numero di clienti è limitato e conosciuto, mentre il throughput è un parametro di output che si ottiene risolvendo il modello.

3.3.3 Modello Misto

Un modello misto contiene sia classi aperte che chiuse. Viene utilizzato per verificare se il sistema può rispettare gli SLO (es. tempi di risposta per le transazioni online) mentre esegue contemporaneamente carichi batch (es. produzione di 20 report/ora).

3.4 Service Level Agreement (SLA)

Gli obiettivi di prestazione e i requisiti di qualità possono essere formalizzati in contratti chiamati Service Level Agreement (SLA) tra il fornitore e il cliente. Uno SLA tipico può includere:

- **Disponibilità:** es. 99.99% in orario lavorativo (8 : 00 – 23 : 00) e 99.9% nel resto del tempo.
- **Tempo di risposta:** es. massimo 4 secondi per lo scaricamento di pagine su connessioni non sicure e 6 secondi per connessioni sicure.
- **Throughput:** es. minimo 2000 pagine al secondo.
- **Penali:** sanzioni previste in caso di mancato raggiungimento dei requisiti.

L'interazione umana introduce ritardi naturali (think time) dovuti alla lettura dei contenuti. Questo può essere rappresentato nel modello come una risorsa di tipo **Delay**: un centro senza coda dove le richieste vengono servite immediatamente, simulando componenti dedicati o situazioni in cui le risorse sono sovrabbondanti rispetto alle richieste.

Al contrario, risorse come una LAN condivisa sono **Load-Dependent**: la velocità di trasmissione percepita dipende dal carico istantaneo generato dagli altri utenti. In questi centri, il tasso di servizio è una funzione del numero di richieste attualmente in coda.

3.5 Admission Control

Ogni server database ha un limite fisico o logico al numero massimo di transazioni concorrenti che può gestire, indicato come W . In PostgreSQL, questo limite è configurabile tramite il parametro `max_connections` (default 100). L'Admission Control agisce come un filtro all'ingresso del sistema: quando una transazione arriva, il sistema verifica se il numero attuale di transazioni è pari a W . Se sì, la transazione viene rifiutata (rejected); altrimenti, viene ammessa all'elaborazione. In presenza di Admission Control, il throughput effettivo si calcola come:

$$\text{Throughput} = \text{Arrival Rate} \times (1 - \text{Prob of Rejection})$$

3.6 Discipline di accodamento

Ogni centro di servizio (tranne i delay) deve gestire l'ordine in cui le richieste in coda vengono servite. Le principali discipline sono:

- **First Come First Served (FCFS)**: le richieste sono servite seguendo l'ordine cronologico di arrivo.
- **Priority Queuing**: viene servito il job con la priorità più alta. Può essere statica o dinamica, con o senza prelazione (preemptive resume per continuare dal punto di interruzione, o preemptive restart per ricominciare).
- **Round Robin (RR)**: ogni transazione riceve un quanto di tempo (time slice); scaduto il quanto, la risorsa passa alla transazione successiva rimettendo la precedente in coda.
- **Processor Sharing (PS)**: versione estrema di Round Robin con quanti di tempo infinitesimali, dando l'illusione di un'esecuzione perfettamente parallela tra tutte le richieste.

3.7 Class Switching

In alcuni scenari, il workload associato a un utente cambia natura nel tempo. Si consideri il processo di autenticazione: un utente inizia in stato "Disconnected", invia una richiesta di autenticazione ("request authentication") passando allo stato "Connected and Authenticating". Se l'autenticazione fallisce ("authentication denied"), torna a "Disconnected". Se ha successo ("successful authentication"), passa allo stato "Connected and Authenticated" per eseguire le interrogazioni al DB. Al termine della transazione ("end of transaction"), torna nuovamente disconnesso. Questo dinamismo viene modellato tramite il **Class Switching**, che aggiunge probabilità di transizione tra classi differenti all'interno della rete di code.

4 Modellamento di sistemi: reti di code (QN)

Una rete di code (Queuing Network, QN) è definita formalmente come una collezione di K code interconnesse tra loro. All'interno di questo framework, i termini "coda" o "stazione" vengono utilizzati per identificare l'unione di due componenti distinte: la vera e propria fila d'attesa (waiting line), dove le richieste sostano in attesa di essere elaborate, e la risorsa fisica o logica (server) che fornisce effettivamente il servizio alla richiesta. Le richieste si muovono dinamicamente all'interno della rete, transitando da una coda all'altra secondo percorsi logici predefiniti, finché non completano la loro esecuzione e abbandonano il sistema. Per fini analitici, le richieste possono essere raggruppate in classi basate sulla similarità del loro comportamento; in scenari avanzati, è possibile modellare situazioni in cui le richieste cambiano classe durante il loro transito nella rete.

4.1 Parametri di input e caratterizzazione del workload

I parametri necessari per definire una coda all'interno di una QN sono suddivisi in due categorie principali: l'intensità del workload e i service demands. I parametri di intensità forniscono una misura del carico gravante sul sistema e vengono espressi in termini di tassi di arrivo (λ) per le classi aperte o in termini di popolazione (N) per le classi chiuse. I service demands ($D_{i,r}$) rappresentano il tempo di servizio totale medio che una specifica risorsa i fornisce a una data classe di richieste r . È importante sottolineare che i service demand sono solitamente considerati indipendenti dal carico del sistema, sebbene esistano eccezioni in cui l'intensità del workload può influenzare il tempo di servizio richiesto.

Le classi di workload si distinguono in base alla natura del loro processo di arrivo e permanenza nel sistema:

- **Transaction:** identifica una classe aperta la cui intensità è rappresentata da un tasso di arrivi esterno (λ). Il numero di utenti nel sistema non è limitato a priori.
- **Batch:** identifica una classe chiusa in cui l'intensità è rappresentata dal numero fisso di clienti (N) che circolano costantemente all'interno della QN. Non appena un job termina, un altro entra immediatamente nel sistema.
- **Interactive:** è una variante della classe chiusa utilizzata per modellare scenari in cui le richieste sono generate da un numero prefissato di macchine client o terminali (M). L'intensità è definita dal numero di terminali M e dal tempo di riflessione medio (think time, Z), ovvero l'intervallo tra la ricezione di una risposta e l'invio della successiva richiesta.

4.2 Notazione formale e metriche di performance

Per descrivere le caratteristiche di una QN si adotta una notazione standardizzata. Ogni coda è specificata dal suo tipo, che può essere indipendente dal carico (load independent, LI), dipendente dal carico (load dependent, LD) o di ritardo (delay), e dalla sua disciplina di accodamento, come First Come First Served (FCFS), Round Robin (RR) o Processor Sharing (PS).

I simboli fondamentali includono:

- K : numero totale di code nella rete.
- R : numero totale di classi di workload.
- $\lambda = (\lambda_1, \dots, \lambda_R)$: vettore dei tassi di arrivo per QN aperte, dove λ_r è il tasso medio per la classe r .
- $N = (N_1, \dots, N_R)$: vettore della popolazione per QN chiuse, dove N_r è il numero di clienti della classe r .
- $D_{i,r}$: service demand della classe r presso la risorsa i .
- $Prior(r)$: livello di priorità della classe r , con valore 1 per la priorità massima.

Dalla risoluzione del modello si ottengono le metriche di output principali: il throughput delle richieste per classe ($X_{0,r}$) e il tempo di risposta medio per classe ($R_{0,r}$).

4.3 Modellamento di un database server: esempio pratico

L'architettura di un server database può essere modellata come una rete di code composta da una CPU e un disco. Lo schema grafico prevede un flusso di "arriving transactions" che entrano in un rettangolo suddiviso lateralmente (la coda della CPU) collegato a un cerchio che rappresenta il servente CPU. All'uscita della CPU, il flusso si biforca: una parte costituisce le "completing transactions" che lasciano il sistema, mentre l'altra si dirige verso la coda del disco (rappresentata in modo analogo con rettangolo e cerchio). Dopo l'elaborazione del disco, la richiesta torna nella coda della CPU, chiudendo il ciclo.

Si consideri un sistema con $K = 2$ (CPU e disco) e $R = 3$ classi aperte: Semplice (45%), Medio (25%) e Complesso (30%). Con un tasso totale $\lambda = 1.5$ tps, i tassi specifici sono:

- $\lambda_{semplice} = 1.5 \times 0.45 = 0.675$ tps
- $\lambda_{medio} = 1.5 \times 0.25 = 0.375$ tps
- $\lambda_{complesso} = 1.5 \times 0.30 = 0.450$ tps

Per calcolare i service demand, si integrano le informazioni sulle visite e sui tempi medi. Se ogni richiesta visita la CPU due volte, e i tempi medi per singola visita sono $0.04s, 0.18s, 1.20s$, avremo $D_{CPU,semplice} = 0.08s, D_{CPU,medio} = 0.36s, D_{CPU,complesso} = 2.4s$. Se un'operazione di I/O dura $0.01s$, i service demand del disco si ottengono moltiplicando tale valore per il numero medio di I/O (5.5, 28.9, 85.0), ottenendo rispettivamente $0.055s, 0.289s, 0.850s$.

4.4 Analisi di workload batch e scenari di sistema

In scenari notturni con due applicazioni batch multithread concorrenti, il modello diventa una QN chiusa. La prima applicazione (report vendite) opera con 5 thread e ha service demand $D_{CPU} = 45s$ e $D_{Disk} = 50s$. La seconda (report marketing) opera con 10 thread con $D_{CPU} = 80s$ e $D_{Disk} = 96s$. In questo caso, l'obiettivo è determinare il throughput atteso per ciascuna applicazione isolando l'interazione tra le risorse.

Ulteriori esercizi permettono di comprendere la sensibilità del sistema ai cambiamenti:

1. **Connessione di rete:** una linea a 20Mbps trasmette pacchetti da 1500 byte. Il service demand è il tempo di trasmissione: $D = 1500 / (20 \times 1024 \times 1024 / 8) = 5.72 \times 10^{-4} s$.
2. **Sistemi misti:** un workload composto da transazioni a 10 tps (aperte) e 50 client interattivi (chiusi) richiede una QN mista per una corretta rappresentazione.
3. **Ottimizzazione hardware e software:** se un disco ($D_1 = 100ms$) viene sostituito da uno più veloce del 40%, il nuovo service demand si calcola basandosi sull'incremento del tasso di servizio: $100ms \rightarrow 10 tps$; $10 tps \times 1.4 = 14 tps$; $D_{nuovo} = 1/14 = 71.4ms$. Se si abilita il log su un secondo disco ($D_2 = 150ms$) per il 30% di operazioni di update (15ms a log), il nuovo demand è $D_{2,nuovo} = 150 + 0.3 \times 15 = 154.5ms$.

4.4.1 Aggregazione delle classi

Quando l'analisi si focalizza su una specifica classe (es. Semplice) o su cambiamenti strutturali (es. spostare le transazioni di Update su un'altra macchina), è possibile semplificare il modello aggregando classi simili. Per fondere le classi "Medio" ($\lambda = 0.3$) e "Complesso" ($\lambda = 0.2$) in una "Nuova Classe", si calcola il nuovo tasso $\lambda_{new} = 0.3 + 0.2 = 0.5 tps$. Il service demand della nuova classe è la media pesata dei demand originali. Ad esempio, per la CPU: $D_{CPU,new} = (0.3/0.5) \times 0.3 + (0.2/0.5) \times 0.45 = 0.36$. Lo stesso principio si applica ai dischi per mantenere la coerenza del modello aggregato.

5 Modellamento di sistemi: aspetti quantitativi

L'analisi dei sistemi informatici richiede strumenti rigorosi per rispondere a interrogativi fondamentali riguardanti le prestazioni al variare del carico di lavoro. Tra le domande più ricorrenti figurano la determinazione del tempo di risposta in funzione del carico, il calcolo del throughput complessivo del sistema e la valutazione del grado di utilizzazione delle risorse disponibili. Inoltre, risulta essenziale prevedere il comportamento del sistema a seguito di modifiche strutturali o hardware. Per rispondere a queste necessità, si adotta un approccio metodologico denominato analisi operativa, il quale permette di comprendere le relazioni matematiche tra le diverse quantità misurabili in gioco.

5.1 Variabili operative e definizioni fondamentali

Il punto di partenza dell'analisi operativa consiste nell'identificazione di variabili che possono essere misurate direttamente durante un periodo di osservazione del sistema. Si definiscono le seguenti quantità fondamentali:

- T : rappresenta la lunghezza totale del periodo di osservazione espresso in unità di tempo (ad esempio, secondi o ore).
- K : indica il numero totale di risorse fisiche o logiche che compongono il sistema oggetto di studio.
- B_i : è il tempo in cui la specifica risorsa i risulta occupata (busy) durante l'intero intervallo di osservazione T .
- A_i : rappresenta il numero totale di richieste di servizio, o arrivi, registrati presso la risorsa i nell'intervallo T .
- A_0 : indica il numero totale di richieste che sono entrate nel sistema complessivo durante il periodo T .
- C_i : esprime il numero totale di richieste di servizio completate ed evase dalla risorsa i nell'intervallo T .
- C_0 : rappresenta il numero totale di richieste che hanno completato il loro intero ciclo di elaborazione e sono state evase dal sistema nel periodo T .

Queste variabili operative costituiscono la base per derivare metriche prestazionali più complesse.

5.1.1 Metriche derivate

A partire dalle grandezze misurate, è possibile calcolare diversi parametri che descrivono l'efficienza e il comportamento del sistema:

1. **Tempo medio di servizio** (S_i): Definito come il rapporto tra il tempo di occupazione della risorsa e il numero di richieste evase dalla stessa, ovvero $S_i = \frac{B_i}{C_i}$. Esso rappresenta il tempo medio richiesto dalla risorsa i per processare una singola richiesta, escludendo il tempo trascorso in coda.
2. **Utilizzazione della risorsa** (U_i): Rappresenta la frazione di tempo in cui la risorsa i è stata attiva rispetto al tempo totale di osservazione, calcolata come $U_i = \frac{B_i}{T}$. Solitamente viene espressa in percentuale.
3. **Throughput della risorsa** (X_i): Indica il tasso di completamento delle richieste per la risorsa i , calcolato come $X_i = \frac{C_i}{T}$.
4. **Tasso degli arrivi alla risorsa** (λ_i): Definito come $\lambda_i = \frac{A_i}{T}$, indica la frequenza con cui nuove richieste giungono alla risorsa i .
5. **Throughput del sistema** (X_0): Rappresenta la produttività globale del sistema, ovvero il numero di transazioni completate nell'unità di tempo, calcolato come $X_0 = \frac{C_0}{T}$.

5.2 Esempio pratico di analisi operativa

Si consideri un database server modellato con una singola CPU. Si supponga che durante un minuto di osservazione ($T = 60s$) si ottengano i seguenti dati: la CPU è rimasta occupata per $36s$ ($B_{CPU} = 36s$), sono state richieste 1800 transazioni ($A_{CPU} = A_0 = 1800$) e ne sono state completate altrettante ($C_{CPU} = C_0 = 1800$).

Per determinare il tempo medio di servizio per transazione presso la CPU, applichiamo la formula: $S_{CPU} = \frac{B_{CPU}}{C_{CPU}} = \frac{36}{1800} = \frac{1}{50} = 0.02s$ per transazione.

L'utilizzazione della risorsa CPU è data da: $U_{CPU} = \frac{B_{CPU}}{T} = \frac{36}{60} = 0.6 = 60\%$.

Il throughput del sistema, che in questo modello semplificato coincide con quello della CPU, è: $X_0 = X_{CPU} = \frac{C_0}{T} = \frac{1800}{60} = 30tps$ (transazioni al secondo).

In scenari più realistici, è fondamentale suddividere il carico di lavoro in classi differenti. La notazione operativa viene estesa introducendo il parametro R per il numero di classi e aggiungendo un indice specifico. Ad esempio, $U_{i,r}$ indica l'utilizzazione della risorsa i imputabile esclusivamente alla classe di workload r , mentre $X_{0,r}$ rappresenta il throughput del sistema riferito alla medesima classe.

5.3 Leggi operative fondamentali

Le leggi operative definiscono le relazioni matematiche stabili tra le variabili operative e le metriche derivate.

5.3.1 Legge dell'utilizzazione (Utilization Law)

Partendo dalla definizione di utilizzazione $U_i = \frac{B_i}{T}$, è possibile moltiplicare numeratore e denominatore per C_i , ottenendo $U_i = \frac{B_i \times C_i}{C_i \times T}$. Riconoscendo le definizioni di S_i e X_i , si ricava la legge fondamentale: $U_i = S_i \times X_i$

Questa legge stabilisce che l'utilizzazione di una risorsa è pari al prodotto tra il tempo medio di servizio e il throughput della risorsa stessa. In condizioni di equilibrio, dove il numero di arrivi è uguale a quello dei completamenti ($A_i = C_i$), ne consegue che $\lambda_i = X_i$ e quindi $U_i = S_i \times \lambda_i$. Nel caso di una risorsa dotata di m server (coda multiprocessore), la formula diventa: $U_i = \frac{S_i \times X_i}{m}$. In un contesto multi-classe, l'utilizzazione per una specifica classe r è $U_{i,r} = \frac{S_{i,r} \times X_{i,r}}{m}$.

Un esempio applicativo riguarda un canale di comunicazione a $56Kbps$ che trasmette pacchetti da $1500bytes$ ($12000bits$) al tasso di $3pps$ (pacchetti al secondo). Identifichiamo il throughput $X_0 = 3pps$. Il tempo di servizio S_0 è il tempo di trasmissione del singolo pacchetto: $S_0 = \frac{12000}{56000} = 0.214s$. L'utilizzazione del link è dunque $U_0 = 0.214 \times 3 = 0.642 = 64.2\%$.

5.3.2 Service Demand e la sua legge

Il Service Demand (D_i) rappresenta il tempo totale medio speso da una singola richiesta presso la risorsa i durante l'intero ciclo di vita nel sistema. Poiché una richiesta può visitare una risorsa più volte prima

di essere completata, il service demand è la somma dei singoli tempi di visita. Per definizione, esso non include il tempo trascorso in coda.

Per calcolare il D_i si utilizza la Service Demand Law: $D_i = \frac{B_i}{C_0} = \frac{U_i \times T}{C_0} = \frac{U_i}{C_0/T} = \frac{U_i}{X_0}$

Questa relazione può essere interpretata anche come $D_i = S_i \times V_i$, dove V_i è il numero medio di visite alla risorsa i per completare una richiesta. In ambito multi-classe, per la risorsa i e la classe r , si ha $D_{i,r} = \frac{U_{i,r}}{X_{0,r}} = S_{i,r} \times V_{i,r}$.

Consideriamo un web server monitorato per 10 minuti (600s) con una CPU occupata al 90% ($U_{CPU} = 0.9$). Se vengono processate 30000 richieste, il throughput è $X_0 = \frac{30000}{600} = 50req/sec$. Il service demand della CPU risulta $D_{CPU} = \frac{0.9}{50} = 0.018sec/req$.

5.3.3 Analisi di un sistema a più dischi

Si analizzi un computer con una CPU (risorsa 1) e tre dischi (risorse 2, 3, 4) che esegue un DBMS sotto carico batch (sistema chiuso). In un periodo di osservazione $T = 1h = 3600s$, vengono completate $C_0 = 13680$ transazioni. Il throughput del sistema è $X_0 = \frac{13680}{3600} = 3.8tps$. Si dispone dei seguenti dati per i dischi:

- Disco 2: 32IOPS, $U_2 = 0.3$
- Disco 3: 36IOPS, $U_3 = 0.41$
- Disco 4: 50IOPS, $U_4 = 0.54$

I tempi medi di servizio per richiesta ($S_i = \frac{U_i}{X_i}$) sono: $S_2 = \frac{0.3}{32} = 0.0094s$; $S_3 = \frac{0.41}{36} = 0.0114s$; $S_4 = \frac{0.54}{50} = 0.0108s$.

I service demand ($D_i = \frac{U_i}{X_0}$), assumendo $U_{CPU} = 35\%$, sono: $D_1 = \frac{0.35}{3.8} = 0.092s$; $D_2 = \frac{0.3}{3.8} = 0.079s$; $D_3 = \frac{0.41}{3.8} = 0.108s$; $D_4 = \frac{0.54}{3.8} = 0.142s$.

5.3.4 Legge del flusso forzato (Forced Flow Law)

Questa legge lega il throughput di una singola risorsa i al throughput complessivo del sistema X_0 tramite il numero medio di visite V_i : $X_i = X_0 \times V_i$

In termini multi-classe, abbiamo $X_{i,r} = X_{0,r} \times V_{i,r}$. Riprendendo l'esempio dei tre dischi con $X_0 = 3.8tps$, il numero medio di visite per ogni disco si calcola come $V_i = \frac{X_i}{X_0}$: $V_2 = \frac{32}{3.8} = 8.4$ visite; $V_3 = \frac{36}{3.8} = 9.5$ visite; $V_4 = \frac{50}{3.8} = 13.2$ visite.

5.4 Legge di Little (Little's Law)

La legge di Little è una relazione estremamente generale che lega il numero medio di clienti nel sistema (N), il tempo medio di permanenza (R) e il tasso di arrivo/completamento (X). Essa è applicabile a qualsiasi "scatola nera" in cui i clienti non vengano né creati né distrutti.

Un esempio intuitivo è quello di un pub: se entra un nuovo cliente ogni ora e la permanenza media è di 3.5 ore, in media ci saranno $3.5 \times 1 = 3.5$ clienti nel pub.

A livello di sistema o di singola risorsa, la legge si declina in diverse forme:

- **Solo server (senza coda)**: Il numero medio di utenti all'interno del server $N_{i,s}$ corrisponde all'utilizzazione del server stesso: $N_{i,s} = U_i = X_i \times S_i$.
- **Centro completo (server + coda)**: $N_i = X_i \times R_i$, dove R_i è il tempo di risposta medio del centro (tempo di servizio + tempo di coda).
- **Sola coda**: $N_{i,w} = X_i \times W_i$, dove W_i è il tempo medio trascorso in coda.
- **Sistema globale**: $N_0 = X_0 \times R_0$, dove R_0 è il tempo di risposta medio delle transazioni.

Si consideri un sistema con un throughput $X_0 = 3.8tps$. Se si misura che in media ci sono 16 transazioni in esecuzione contemporanea ($N_0 = 16$), il tempo di risposta medio del sistema è: $R_0 = \frac{N_0}{X_0} = \frac{16}{3.8} = 4.2s$.

6 Legge del tempo di risposta interattivo e algoritmi di analisi

Nei sistemi informatici caratterizzati da un'interazione diretta con l'utente, come i database interattivi, il carico di lavoro viene modellato considerando un numero fisso di utenti che operano tramite workstation. Questo scenario viene rappresentato graficamente da un insieme di M postazioni client (identificate dai nodi numerati da 1 a M) collegate a un blocco centrale denominato "Interactive Database System". Il flusso è ciclico: una richiesta esce dal client, entra nel sistema database, viene processata e la risposta torna al client.

All'interno di questo modello, gli utenti alternano due fasi distinte:

- **Think time (Z):** rappresenta il tempo medio durante il quale l'utente prepara la query successiva dopo aver ricevuto la risposta alla precedente.
- **Response time (R):** denota il tempo di risposta medio, ovvero l'intervallo durante il quale l'utente è in attesa di una risposta dal database.

Per analizzare il sistema, definiamo $\bar{M}$ come il numero medio di utenti che si trovano nella fase di riflessione (think phase) e $\bar{N}$ come il numero medio di utenti in attesa di risposta. La popolazione totale dei client è data dalla somma dei due stati: $M = \bar{M} + \bar{N}$. Applicando la Legge di Little a ciascuna fase, otteniamo che $\bar{M} = X_0 \times Z$ (dove X_0 è il throughput del sistema) e $\bar{N} = X_0 \times R$. Sostituendo queste espressioni nell'equazione della popolazione totale, si ricava $M = X_0 \times Z + X_0 \times R$, che può essere riscritta come $M = X_0(Z + R)$. Attraverso passaggi algebrici, si giunge alla formulazione finale della **Legge del tempo di risposta interattivo**: $R = \frac{M}{X_0} - Z$

6.1 Esempi applicativi delle leggi operazionali

6.1.1 Esempio 7: Calcolo del tempo di risposta medio

Si consideri un sistema monitorato per un'ora in cui vengono evase 7200 richieste inviate da 40 client con un think time medio di 15 secondi. Per calcolare il tempo di risposta medio R , determiniamo prima il throughput del sistema: $X_0 = \frac{7200}{3600} = 2$ req/s. Applicando la legge del tempo di risposta interattivo: $R = \frac{40}{2} - 15 = 20 - 15 = 5s$

6.1.2 Esempio 8: Calcolo del tempo di risposta con vincoli sulle risorse

In un sistema con 150 client ($M = 150$) e un think time medio di 10s ($Z = 10s$), si misurano i seguenti dati relativi al disco: utilizzazione media $U_{DISK} = 50\%$ (0.5), numero medio di accessi per richiesta $V_{DISK} = 2$ e tempo medio di servizio del disco $S_{DISK} = 25ms$ (0.025s). L'obiettivo è determinare R . Dalla legge dell'utilizzazione ricaviamo il throughput del disco: $X_{DISK} = \frac{U_{DISK}}{S_{DISK}} = \frac{0.5}{0.025} = 20$ req/s. Dalla legge del flusso forzato ricaviamo il throughput del sistema: $X_0 = \frac{X_{DISK}}{V_{DISK}} = \frac{20}{2} = 10$ req/s. Otteniamo infine il tempo medio di risposta: $R = \frac{150}{10} - 10 = 15 - 10 = 5s$

6.2 Riepilogo delle leggi fondamentali (In sintesi)

- **Utilization Law:** $U_i = X_i \times S_i = \lambda_i \times S_i$
- **Forced Flow Law:** $X_i = V_i \times X_0$
- **Service Demand Law:** $D_i = V_i \times S_i = \frac{U_i}{X_0}$
- **Little's Law:** $N = X \times R$
- **Interactive Response Time Law:** $R = \frac{M}{X_0} - Z$

6.3 Esercitazione pratica sui modelli di code

6.3.1 Esercizio 1: Applicazione diretta della Legge di Little

Un sistema monitorato per 1h completa 7200 transazioni con una media di 5 transazioni concorrenti presenti. Il tempo medio di risposta è: $R = \frac{N}{X} = \frac{5}{7200/3600} = \frac{5}{2} = 2.5s$

6.3.2 Esercizio 4: Calcolo del Service Demand

In un monitoraggio di 30 minuti (1800s) vengono completate 5400 transazioni e 18900 operazioni di I/O su un disco con utilizzazione al 40%. Il numero medio di operazioni di I/O per transazione è:

$V_i = \frac{18900}{5400} = 3.5$ I/O/transazione Il tempo di servizio medio del disco per singola operazione è: $S_i = \frac{U_i}{X_i} = \frac{0.4}{18900/1800} = \frac{0.4}{10.5} = 0.038$ s/IO Il service demand totale per una transazione sulla risorsa disco è: $D_i = V_i \times S_i = 3.5 \times 0.038 = 0.133s$

6.3.3 Esercizio 6: Numero medio di pacchetti in rete

Dato un ritardo medio di $100ms$ ($0.1s$) e un transito di 128 pkt/s, il numero medio di pacchetti che transitano contemporaneamente sulla rete è: $N = X \times R = 128 \times 0.1 = 12.8$

6.4 Mean Value Analysis (MVA)

L'algoritmo Mean Value Analysis è una tecnica ricorsiva utilizzata per risolvere reti di code chiuse. Esso permette di calcolare il tempo medio di residenza in ogni centro, il tempo di risposta e il throughput del sistema, il throughput e l'utilizzazione dei singoli centri, nonché il numero medio di job in ogni risorsa.

6.4.1 Algoritmo MVA

1. Si inizializza il numero medio di job per ogni centro i a zero per una popolazione nulla: $n_i(0) = 0$.
2. Per ogni valore di popolazione n da 1 a N :
 1. Calcolo del tempo medio di residenza per ogni centro i : $R'_i(n) = V_i S_i + V_i S_i n_i(n-1) = D_i [1 + n_i(n-1)]$.
 2. Calcolo del tempo di risposta complessivo: $R_0(n) = \sum_{i=1}^K R'_i(n)$.
 3. Calcolo del throughput del sistema (Legge di Little): $X_0(n) = \frac{n}{R_0(n)}$.
 4. Calcolo del throughput di ogni centro i (Legge del flusso forzato): $X_i(n) = V_i \times X_0(n)$.
 5. Calcolo dell'utilizzazione di ogni centro i (Legge dell'utilizzazione): $U_i(n) = S_i \times X_i(n)$.
 6. Calcolo del numero medio di job per ogni centro i (Legge di Little): $n_i(n) = X_i(n) \times R'_i(n)$ (oppure $X_0(n) \times R'_i(n)$).

6.4.2 Applicazione dell'MVA: Esercizio 8

Si consideri un sistema composto da una CPU (centro 1) e tre dischi (centri 2, 3, 4) disposti in parallelo dopo l'uscita dalla CPU. I service demand forniti sono: $D_{CPU} = 0.092s$, $D_{DISK1} = 0.079s$, $D_{DISK2} = 0.108s$, $D_{DISK3} = 0.142s$. Supponendo che ci siano 5 transazioni in esecuzione, analizziamo il primo passo dell'algoritmo ($n = 1$):

1. Inizializzazione: $n_{CPU}(0) = n_{DISK1}(0) = n_{DISK2}(0) = n_{DISK3}(0) = 0$.
2. Calcolo per $n = 1$:
 - Tempi di residenza: $R'_{CPU}(1) = 0.092(1+0) = 0.092s$; $R'_{DISK1}(1) = 0.079s$; $R'_{DISK2}(1) = 0.108s$; $R'_{DISK3}(1) = 0.142s$.
 - Tempo di risposta complessivo: $R_0(1) = 0.092 + 0.079 + 0.108 + 0.142 = 0.421s$.
 - Throughput del sistema: $X_0(1) = \frac{1}{0.421} = 2.375$ tps.
 - Throughput dei device: $X_{CPU}(1) = X_{DISK1}(1) = X_{DISK2}(1) = X_{DISK3}(1) = 2.375$ tps (assumendo $V_i = 1$ per tutti i dispositivi rispetto al sistema).
 - Utilizzazioni ($U = S \times X$): $U_{CPU}(1) = 0.092 \times 2.375 = 0.21$; $U_{DISK1}(1) = 0.18$; $U_{DISK2}(1) = 0.25$; $U_{DISK3}(1) = 0.33$.
 - Numero medio di job: $n_{CPU}(1) = 2.375 \times 0.092 = 0.21$ job; $n_{DISK1}(1) = 0.18$ job; $n_{DISK2}(1) = 0.25$ job; $n_{DISK3}(1) = 0.33$ job.